Software Requirements Specification

AI-Powered Software Bug Investigation and Resolution Platform

Version: 2.0
Date: October 2026
Institution: University of Engineering and Technology (UET), Lahore
Department: Computer Science
Supervisor: Dr. Amjad Farooq
Team:

- Mobeen Butt (2023-CS-28)
- Muhammad Kaleem Faryad (2023-CS-158)
- Sehrish Saif (2023-CS-34)

Table of Contents

1. Introduction
2. Overall Description
3. Functional Requirements
    - 3.1 Incident Ingestion & Webhook
    - 3.2 Orchestrator
    - 3.3 Code Agent
    - 3.4 Log Agent
    - 3.5 Database Agent
    - 3.6 History & RAG Agent

    - 3.7 Verifier & Hypothesis Engine
    - 3.8 Fix Agent
    - 3.9 Test Agent
    - 3.10 Sandbox Testing
    - 3.11 Dashboard
    - 3.12 Integrations
    - 3.13 n8n Automation
    - 3.14 Evaluation Framework
    - 3.15 Feedback Loop
4. External Interface Requirements
5. Non-Functional Requirements
6. Use Cases
7. Domain Data Model
8. Security Requirements
9. Constraints and Assumptions
10. Out of Scope
11. Glossary

---

1. Introduction

1.1 Purpose

This document specifies the complete software requirements for the AI-Powered Software Bug Investigation and Resolution Platform. It is written for the project team, the FYP supervisor, and any future contributors. It serves as the authoritative reference for all implementation decisions.

1.2 Project Overview

The platform accepts a software incident from any trigger source and autonomously runs a full investigation pipeline: evidence collection from code, logs, and the database; root-cause analysis with confidence scoring; reproduction test generation; fix generation and sandbox

verification; and a human-gated approval workflow that produces a draft pull request. Human approval is always required before any code change is merged or deployed.

The platform's defining characteristics are:

1. **Evidence-linked claims** — every statement in a report is traceable to a real log line, commit, or database row. Unsupported claims are rejected by the Verifier agent.
2. **Reproduce-before-fix** — the system first writes a test that fails because of the bug. A fix is only accepted if that test passes after the patch and no existing test breaks.
3. **Hypothesis testing with execution proof** — each root-cause hypothesis is verified by running a controlled experiment in the sandbox (e.g., git bisect, targeted input injection), not just inferred by the language model.
4. **Active evidence gathering** — if evidence is insufficient, the system autonomously fetches more or asks the developer a targeted question before proceeding.
5. **Organizational memory** — past incidents, fixes, and rejections are stored in a Knowledge Graph and RAG layer so the system improves on recurring patterns.

1.3 Scope

The platform covers the complete workflow from incident trigger to approved draft pull request. It supports:

- **Languages:** Python (full support), JavaScript/TypeScript (full support)
- **Source control:** GitHub (primary), GitLab (secondary)
- **Databases:** PostgreSQL (read-only)
- **Autonomy levels:** Level 1 (investigation report only) and Level 2 (suggested fix with draft PR)
- **Clients:** Web dashboard (primary), VS Code extension (optional), n8n automation layer

1.4 Definitions

Term	Definition
Incident	A reported software failure: a failed CI build, an error alert, a Jira ticket, or a manual submission
Evidence	A verifiable artifact linked to an incident: a log line, a commit, a database row, a test result
Claim	A statement in a report, linked to one or more evidence items and a confidence score
Hypothesis	A candidate root cause with supporting and contradicting evidence and an overall score
Agent	A component that collects and structures one type of evidence or performs one action
Orchestrator	The LangGraph-based workflow engine that coordinates all agents and manages incident state
Sandbox	An isolated Docker container in which fixes are tested without touching production
Reproduction test	A test written to demonstrate the bug; it must fail before the fix and pass after
Regression test	A test that confirms the fix does not break existing behaviour
Knowledge Graph	A graph database storing relationships between incidents, causes, fixes, and code entities
RAG	Retrieval-Augmented Generation — using vector search over past incidents to inform current investigation
Confidence score	A value between 0 and 1 representing how well evidence supports a hypothesis
Flaky test	A test that produces inconsistent results across multiple identical runs
Webhook	An HTTP call sent by an external system when an event occurs
Postmortem	A structured document summarising an incident after resolution

n8n	A workflow automation tool used to route events between GitHub, Jira, Sentry, Slack, and the platform

2. Overall Description

2.1 Product Perspective

The platform is a standalone SaaS application. All modules share a single FastAPI backend. The dashboard is a React client of that backend. External systems (GitHub, Jira, Sentry, Slack) interact through webhooks and REST APIs, mediated by n8n automation workflows. The orchestrator drives all agents. The evaluation framework runs the full pipeline on test cases offline.

2.2 Incident Lifecycle (State Machine)

```
RECEIVED → VALIDATED → TRIAGED → INVESTIGATING → EVIDENCE
→ HYPOTHESIS_READY → VERIFIED | UNCERTAIN → REPRODUCED
→ FIX_GENERATED → SANDBOX_TESTING → READY_FOR_APPROVAL
→ APPROVED | REJECTED → DRAFT_PR_CREATED → RESOLVED
```

Any state can transition to `FAILED` or `NEEDS_HUMAN_INPUT`. The orchestrator owns all state transitions.

2.3 User Classes

User	Description
Developer	Reviews evidence, diffs, and test results; approves or rejects fixes
QA Engineer	Uses reproduction steps and regression tests

DevOps / SRE	Connects CI/CD and monitoring sources; manages integrations
Manager	Reads plain-language reports, cost summaries, and recurring-issue trends
Evaluator	Team member who runs evaluation experiments and reads metrics

2.4 Assumptions

- The target application uses PostgreSQL.
- The user can supply a repository link and a read-only database connection string.
- Docker is available on the deployment machine.
- A GitHub account with webhook and API access is available.
- A Jira project is available (free plan minimum).

---

3. Functional Requirements

Priority key: M = Must Have | S = Should Have | C = Could Have

---

3.1 Incident Ingestion & Webhook

ID	Requirement	Priority
FR-ING-1	The system shall receive GitHub Actions failure events through a signed webhook and automatically create an incident.	M
FR-ING-2	The system shall receive Jira issue creation and status-change events through a webhook and create or update an incident.	M

FR-ING-3	The system shall receive Sentry error events through a webhook and create an incident.	S
FR-ING-4	The system shall verify the HMAC signature of every incoming webhook and reject unsigned or invalid requests with HTTP 401.	M
FR-ING-5	The system shall reject duplicate webhook events using an idempotency key and return HTTP 200 without creating a duplicate incident.	M
FR-ING-6	The system shall accept a manually submitted incident through a form with a title, description, severity, repository URL, and optional file attachments.	M
FR-ING-7	The system shall auto-classify the severity of an incoming incident (critical, high, medium, low) based on source type, error message, and affected service.	S
FR-ING-8	The system shall validate that a submitted incident references an accessible repository before passing it to the orchestrator.	M
FR-ING-9	The system shall log every received event with source, timestamp, and processing outcome.	M

3.2 Orchestrator

ID	Requirement	Priority
FR-ORC-1	The system shall manage the full incident state machine from RECEIVED to RESOLVED, persisting every state transition with a timestamp.	M
FR-ORC-2	The system shall run the Code, Log, Database, and History agents in parallel during the INVESTIGATING phase and wait for all to complete before proceeding.	M

FR-ORC-3	The system shall check evidence sufficiency after the parallel collection phase. If evidence is insufficient, it shall autonomously fetch additional evidence from the most relevant source or pause the investigation and ask the developer a targeted question.	M
FR-ORC-4	The system shall execute hypothesis testing with execution proof: for each top-ranked hypothesis, it shall design and run a controlled experiment in the sandbox (git bisect, targeted input injection, or equivalent) and mark the hypothesis as verified or rejected based on the experiment result — not solely on LLM inference.	M
FR-ORC-5	The system shall enforce the reproduce-before-fix rule: the Fix Agent is not called unless a reproduction test exists and has been confirmed to fail in the sandbox.	M
FR-ORC-6	The system shall handle agent failure gracefully: if any agent fails, the incident is saved with the available evidence and the failure is recorded; investigation continues with remaining agents.	M
FR-ORC-7	The system shall transition an incident to NEEDS_HUMAN_INPUT when evidence is insufficient after active gathering, and notify the developer via the dashboard.	M
FR-ORC-8	The system shall enforce a configurable investigation timeout (default 30 minutes); on expiry it saves all collected evidence and transitions to NEEDS_HUMAN_INPUT.	M
FR-ORC-9	The system shall route LLM calls by complexity: use a low-cost model (Gemini Flash or equivalent) for triage and classification, and a high-capability model for root-cause reasoning and fix generation. The cost of each model call shall be tracked and summed per incident.	S

FR-ORC-10	The system shall support a privacy mode in which all LLM calls are routed to a locally hosted model (Ollama / vLLM) instead of an external API, with no code or log data leaving the deployment machine.	S
FR-ORC-11	The system shall produce a causal chain timeline: a structured, ordered sequence of events (config change → deploy → error → symptom) with each step linked to at least one evidence item.	S
FR-ORC-12	The system shall generate three versions of the incident report from the same evidence: a technical report for developers, a plain-language impact summary for managers, and a brief customer-facing explanation for support teams.	S

3.3 Code Agent

ID	Requirement	Priority
FR-CODE-1	The system shall clone or fetch the target repository at the commit associated with the incident.	M
FR-CODE-2	The system shall identify all files changed in the commit(s) related to the incident, with diffs.	M
FR-CODE-3	The system shall parse Python and JavaScript/TypeScript source files using tree-sitter to extract function signatures, call graphs, imports, and class hierarchies.	M
FR-CODE-4	The system shall trace the call path from the error entry point to the suspected faulty function.	M

FR-CODE-5	The system shall retrieve git blame information to identify which commit last modified each relevant line.	M
FR-CODE-6	The system shall operate read-only on the repository; it shall never commit, push, or modify files.	M
FR-CODE-7	The system shall detect if the incident-related commit touches a module that has caused incidents in the past and flag it.	S
FR-CODE-8	The system shall support JavaScript/TypeScript in addition to Python for all code parsing operations.	M

3.4 Log Agent

ID	Requirement	Priority
FR-LOG-1	The system shall accept log input from an uploaded file, a configured log directory path, or a Loki/Sentry log source.	M
FR-LOG-2	The system shall parse both structured (JSON) and unstructured (plaintext) log formats.	M
FR-LOG-3	The system shall extract stack traces and correlate them with source file locations.	M
FR-LOG-4	The system shall identify deployment boundaries in the log stream (start/stop events, version strings) to correlate with incident timing.	M
FR-LOG-5	The system shall filter log entries by severity (ERROR, WARN, FATAL) and by a configurable time window around the incident timestamp.	M

FR-LOG-6	The system shall return log evidence in a structured format with timestamps so the orchestrator can place events on the causal timeline.	M
FR-LOG-7	The system shall mask credentials, tokens, and PII patterns in log content before storing or sending to an LLM.	M

3.5 Database Agent

ID	Requirement	Priority
FR-DB-1	The system shall connect to the target PostgreSQL database using a read-only account supplied by the user.	M
FR-DB-2	The system shall reject any query that is not a SELECT statement before it reaches the database.	M
FR-DB-3	The system shall read the full schema (tables, columns, types, constraints, indexes) and make it available to other agents.	M
FR-DB-4	The system shall detect schema changes (new, altered, or dropped columns) between two configurable timestamps.	S
FR-DB-5	The system shall check columns related to an incident for NULL values, type mismatches, and constraint violations.	M
FR-DB-6	The system shall limit each query to a configurable maximum execution time (default 10 seconds) and maximum row count (default 1000) to protect the production database.	M
FR-DB-7	The system shall mask sensitive columns (passwords, tokens, emails, PII) before any data is displayed or sent to an LLM.	M

FR-DB-8	The system shall return findings in a structured format with timestamps so the orchestrator can place database events on the causal timeline.	M
FR-DB-9	The system shall record every query it executes, with the query text, row count, and execution time, in an audit log.	S
FR-DB-10	The system shall return its findings within 60 seconds per incident (target; measured in evaluation).	S

3.6 History & RAG Agent

ID	Requirement	Priority
FR-RAG-1	The system shall index all resolved incidents, including evidence items, root causes, fixes applied, and developer feedback, in a vector store using semantic embeddings.	M
FR-RAG-2	The system shall retrieve past incidents similar to the current one using hybrid search (keyword + vector) with reranking.	M
FR-RAG-3	The system shall return each retrieved result with a source locator (incident ID, field, timestamp) so every historical claim is traceable.	M
FR-RAG-4	The system shall maintain a Knowledge Graph of relationships between: incidents, root causes, code modules, commits, fix types, and recurrence patterns.	M
FR-RAG-5	The system shall use the Knowledge Graph to identify whether the current incident matches a known recurring pattern and surface that match in the report.	M

FR-RAG-6	The system shall update the Knowledge Graph and vector index after every resolved incident (success or rejection).	M
FR-RAG-7	The system shall compare keyword-only, vector-only, and hybrid-plus-graph retrieval strategies in the evaluation framework to measure retrieval quality.	S
FR-RAG-8	The system shall flag when an incident touches a module that has caused three or more past incidents, and include a module fragility score in the report.	S

3.7 Verifier & Hypothesis Engine

ID	Requirement	Priority
FR-VER-1	The system shall generate a ranked list of root-cause hypotheses from the collected evidence, with each hypothesis scored by the weight and quality of its supporting evidence.	M
FR-VER-2	The system shall check every claim in the report against its cited evidence. Claims that cannot be traced to a real evidence item shall be removed or marked unsupported.	M
FR-VER-3	The system shall assign a confidence score (0–1) to each hypothesis and to the overall report.	M
FR-VER-4	The system shall record the unsupported-claim rate (number of claims rejected by the Verifier divided by total claims generated) for every investigation.	M

FR-VER-5	The system shall signal UNCERTAIN when the top hypothesis score is below a configurable threshold (default 0.5) and no hypothesis has significantly more support than the others, rather than presenting a low-confidence cause as definitive.	M
FR-VER-6	The system shall list contradicting evidence alongside supporting evidence for each hypothesis.	S
FR-VER-7	The system shall record calibration data: for each confidence bracket (0.0–0.2, 0.2–0.4, … 0.8–1.0), the fraction of investigations where the stated root cause was confirmed correct.	S

3.8 Fix Agent

ID	Requirement	Priority
FR-FIX-1	The system shall generate a fix only after a verified root cause and a confirmed reproduction test exist.	M
FR-FIX-2	The system shall generate the smallest reasonable patch: it shall not refactor unrelated code or change files unrelated to the root cause.	M
FR-FIX-3	The system shall write the fix to an isolated branch or worktree and never modify the main branch directly.	M
FR-FIX-4	The system shall include a rationale document with the fix explaining: the root cause addressed, the change made, and why no smaller change suffices.	M
FR-FIX-5	The system shall support fix generation for Python and JavaScript/TypeScript projects.	M

FR-FIX-6	The system shall track the cost of all LLM calls made during fix generation and include it in the per-incident cost total.	S

3.9 Test Agent

ID	Requirement	Priority
FR-TEST-1	The system shall write a reproduction test that directly exercises the faulty code path identified in the root cause.	M
FR-TEST-2	The system shall confirm, inside the sandbox, that the reproduction test fails against the unfixed code before proceeding to fix generation.	M
FR-TEST-3	The system shall write regression tests that cover the fix and the surrounding code paths.	M
FR-TEST-4	The system shall run tests using pytest for Python projects and Jest or Vitest for JavaScript/TypeScript projects.	M
FR-TEST-5	The system shall report pass/fail with test name, duration, and failure message for every test run.	M
FR-TEST-6	The system shall record evidence precision for each test: whether the test exercises the exact code path the evidence pointed to.	S

3.10 Sandbox Testing

ID	Requirement	Priority
FR-SB-1	The system shall create an isolated Docker container from the repository at the commit associated with the incident.	M

FR-SB-2	The system shall apply a proposed patch inside the sandbox only, never to the original repository.	M
FR-SB-3	The system shall run the existing test suite before and after the patch and compare the results.	M
FR-SB-4	The system shall run newly generated reproduction and regression tests inside the sandbox.	M
FR-SB-5	The system shall mark a fix as verified only if: the reproduction test fails before the patch, passes after the patch, and no previously passing test now fails.	M
FR-SB-6	The system shall block all network access and enforce configurable CPU, memory, and wall-clock limits inside the sandbox (defaults: 2 CPU cores, 2 GB RAM, 10 minutes).	M
FR-SB-7	The system shall capture all test output (stdout, stderr, exit code) and make it available on the incident dashboard.	M
FR-SB-8	The system shall delete the sandbox container and all its filesystem state after the run finishes or times out.	S
FR-SB-9	The system shall report a clear, structured failure reason when a sandbox run does not complete: build error, dependency error, timeout, test failure, or resource limit exceeded.	S
FR-SB-10	The system shall re-run a failing test a configurable number of times (default 5) before treating it as a confirmed failure.	M
FR-SB-11	The system shall classify a test as flaky if it both passes and fails across the re-runs, and as a confirmed failure if it fails on every run.	M
FR-SB-12	The system shall mark an incident as flaky-only and skip the full root-cause investigation pipeline if all failing tests are classified as flaky.	M

FR-SB-13	The system shall display the per-test pass/fail rerun counts on the incident detail page.	S
FR-SB-14	The system shall store flaky test signatures in the knowledge base so repeated flakiness on the same test is recognised in future investigations.	C
FR-SB-15	The system shall support git bisect execution inside the sandbox to identify the exact commit that introduced a regression, as part of hypothesis testing with execution proof.	M

3.11 Dashboard

ID	Requirement	Priority
FR-DASH-1	The system shall let a user sign in and see only the projects they belong to.	M
FR-DASH-2	The system shall show an incident list with title, source, severity, status, and creation time.	M
FR-DASH-3	The system shall let a user filter incidents by status, severity, and source.	S
FR-DASH-4	The system shall show an incident detail page with the root-cause ranking, a confidence score per hypothesis, and the UNCERTAIN signal when confidence is low.	M
FR-DASH-5	The system shall link every claim in a report to the specific evidence item that supports it (log line, commit hash, database row). Clicking a link opens the source evidence.	M
FR-DASH-6	The system shall show a causal chain timeline for each incident: an ordered, visual sequence of events (config change → deploy → error → symptom) with each node linked to its evidence.	S

FR-DASH-7	The system shall display the proposed code diff and the sandbox test results side by side on the approval screen.	M
FR-DASH-8	The system shall let a developer approve or reject a proposed fix, record a reason category (wrong cause, incomplete fix, style, other), and record who decided and when.	M
FR-DASH-9	The system shall let a developer add free-text feedback on any investigation result; feedback is stored in the knowledge base.	S
FR-DASH-10	The system shall let a user submit an incident manually with a description, severity, repository URL, and optional log file attachments.	M
FR-DASH-11	The system shall show live agent status during an investigation using WebSocket streaming: which agents are running, completed, or failed.	M
FR-DASH-12	The system shall generate a postmortem document for a resolved incident containing: summary, causal timeline, root cause with confidence, evidence list, fix applied, and suggested prevention steps.	S
FR-DASH-13	The system shall let a developer edit the postmortem before it is finalised.	S
FR-DASH-14	The system shall export the postmortem as a PDF or Markdown file.	S
FR-DASH-15	The system shall display three report views for each incident: technical (for developers), impact summary (for managers), and customer-facing explanation (for support teams).	S
FR-DASH-16	The system shall show the per-incident investigation cost (total LLM + API spend in USD) on the incident detail page.	S

FR-DASH-17	The system shall show a codebase health dashboard with: modules ranked by incident frequency, MTTD, MTTR, and a fragility trend over time.	S
FR-DASH-18	The system shall show the pass/fail rerun counts for flaky test classification on the incident page.	S
FR-DASH-19	The system shall notify the developer via in-app notification and optionally via email when an incident reaches READY_FOR_APPROVAL or NEEDS_HUMAN_INPUT.	S

3.12 Integrations

ID	Requirement	Priority
FR-INT-1	The system shall receive GitHub Actions failure events through a signed webhook and create an incident automatically.	M
FR-INT-2	The system shall fetch commits, changed files, and diffs from GitHub using the GitHub REST API.	M
FR-INT-3	The system shall create a draft pull request on GitHub containing an approved fix, linked to the incident.	S
FR-INT-4	The system shall create a Jira issue when an incident is opened, update it as the incident progresses, and close it when resolved.	M
FR-INT-5	The system shall receive Jira issue creation and transition events via webhook and create or update incidents accordingly.	M
FR-INT-6	The system shall import application logs from a file upload or a configured log directory.	M

FR-INT-7	The system shall accept Sentry error events via webhook and create incidents from them.	S
FR-INT-8	The system shall store all integration credentials encrypted at rest and never return credential values through the API after they are saved.	M
FR-INT-9	The system shall display the connection status of each configured integration and show a clear error message when a connection fails.	S
FR-INT-10	The system shall send a Slack notification when an incident reaches READY_FOR_APPROVAL or is RESOLVED, if a Slack integration is configured.	S
FR-INT-11	The system shall support GitLab as an alternative source control provider for webhook events and API access.	C

3.13 n8n Automation

ID	Requirement	Priority
FR-N8N-1	The system shall provide an n8n workflow that receives a GitHub Actions failure event and calls the platform's incident ingestion API.	M
FR-N8N-2	The system shall provide an n8n workflow that receives a Jira webhook event and calls the platform's incident ingestion API.	M
FR-N8N-3	The system shall provide an n8n workflow that, on incident resolution, updates the linked Jira issue status and posts a Slack summary message.	M
FR-N8N-4	The system shall provide an n8n workflow that, when a fix is approved, triggers the GitHub draft PR creation and notifies the developer via Slack.	S

FR-N8N-5	The system shall provide an n8n workflow that receives a Sentry alert and creates an incident via the ingestion API.	S
FR-N8N-6	All n8n workflow definitions shall be version-controlled in the repository and deployable without manual reconfiguration beyond environment variable substitution.	M

3.14 Evaluation Framework

ID	Requirement	Priority
FR-EV-1	The system shall load a structured test case set containing real open-source bugs (BugsInPy, SWE-bench subset) and sample-application bugs, each with ground-truth root cause, faulty file, faulty function, and fix commit.	M
FR-EV-2	The system shall run the full investigation pipeline on each test case in a controlled, repeatable manner and store the complete output.	M
FR-EV-3	The system shall compute the following metrics per run: root-cause accuracy, fix success rate (reproduction + regression pass), investigation time, and cost per incident.	M
FR-EV-4	The system shall compute the unsupported-claim rate: the fraction of claims rejected by the Verifier per investigation.	M
FR-EV-5	The system shall compute confidence calibration error: for each confidence bracket, the fraction of cases where the stated root cause was correct.	S
FR-EV-6	The system shall compute evidence precision and recall: what fraction of cited evidence items were genuinely relevant to the root cause.	S

FR-EV-7	The system shall run the same test cases against three baselines and record results: (a) manual investigation, (b) a plain LLM with no tools, (c) keyword-only retrieval.	M
FR-EV-8	The system shall run retrieval ablations comparing keyword-only, vector-only, and hybrid-plus-graph retrieval on evidence relevance metrics.	S
FR-EV-9	The system shall run hypothesis-engine ablations comparing: full system, system without Verifier, system without Knowledge Graph, and system without reproduction-first enforcement.	S
FR-EV-10	The system shall collect structured developer ratings (clarity and usefulness, 1–5) from reviewers for each investigation report.	S
FR-EV-11	The system shall measure flaky test detection accuracy: true positive rate (flaky correctly identified) and false positive rate (real failure wrongly labelled flaky).	S
FR-EV-12	The system shall export all results as formatted tables and charts suitable for inclusion in the final thesis report.	S
FR-EV-13	The system shall list every failed test case with the reason for failure so they can be written up in the thesis limitations section.	M
FR-EV-14	The system shall measure time-to-first-correct-hypothesis: how many minutes elapsed before the system first identified the correct root cause during an investigation.	S

3.15 Feedback Loop

ID	Requirement	Priority

FR-FB-1	The system shall lower the confidence score assigned to fix types that have been rejected repeatedly (default threshold: 3 rejections) in the same project.	S
FR-FB-2	The system shall display to the developer when a confidence score was reduced due to past rejections, with a count of prior rejections.	S
FR-FB-3	The system shall store all developer feedback (free-text and approval reason categories) in the knowledge base and make it available to the History Agent for future investigations.	M
FR-FB-4	The system shall use past fix rejections to refine future hypothesis ranking: fix types that were previously rejected for a given error pattern shall be ranked lower.	S

---

4. External Interface Requirements

4.1 User Interface

A web dashboard with the following pages:

- Login / project selection
- Incident list with filters
- Incident detail (evidence, timeline, hypotheses, diff, test results)
- Approval screen (diff + test results side by side)
- Integration settings
- Evaluation results
- Codebase health dashboard

The dashboard shall use WebSocket connections to stream live agent status during active investigations. It shall be responsive and work on current desktop browsers (Chrome, Firefox, Edge, Safari).

4.2 VS Code Extension (Optional)

A thin client over the same backend API. Allows developers to review and approve incidents without leaving the editor. No separate backend logic; all operations are delegated to the FastAPI backend. Built only if time permits after core dashboard completion.

4.3 Software Interfaces

System	Interface
FastAPI backend	REST (JSON) + WebSocket
GitHub	Webhooks (HMAC-verified) + REST API v3
Jira	Webhooks + REST API
Sentry	Webhooks
Slack	Incoming webhooks
PostgreSQL	Read-only connection via SQLAlchemy
Docker	Python SDK
n8n	REST API + webhook nodes
Ollama / vLLM	OpenAI-compatible REST API (for privacy mode)

---

5. Non-Functional Requirements

ID	Category	Requirement
NFR-1	Performance	The dashboard shall load the incident list within 3 seconds for up to 500 incidents.
NFR-2	Performance	A sandbox run shall complete within a configurable maximum time (default 10 minutes).

NFR-3	Performance	The Database Agent shall return findings within 60 seconds per incident (target; measured in evaluation).
NFR-4	Performance	The full investigation pipeline shall complete within 30 minutes for a medium-complexity incident.
NFR-5	Security	All external traffic shall use HTTPS.
NFR-6	Security	All credentials and secrets shall be stored encrypted at rest (AES-256 or equivalent).
NFR-7	Security	The Database Agent shall have no write access to the target database; this is enforced at the connection level and by query inspection.
NFR-8	Security	The sandbox shall have no access to production systems, external networks, or host credentials.
NFR-9	Security	All webhook signatures shall be verified before processing; unverified requests shall be rejected.
NFR-10	Reliability	If any agent or integration fails, the incident shall be saved with available evidence and the failure shown to the user; the system shall not crash.
NFR-11	Reliability	The system shall handle concurrent investigations without data corruption (target: 10 simultaneous incidents).
NFR-12	Usability	A new developer shall be able to review an incident and approve or reject a fix without prior training (verified in the reviewer study).
NFR-13	Maintainability	Each module shall expose a documented interface (API contract or abstract base class) so it can be replaced or extended without modifying other modules.

NFR-14	Privacy	Sensitive data (PII, credentials, tokens) shall be masked before it is sent to any external LLM service.
NFR-15	Privacy	Privacy mode shall ensure no repository code, log content, or database data leaves the deployment machine when enabled.
NFR-16	Auditability	Every approval, rejection, database query, sandbox run, and agent action shall be logged with the user (or system) identity and timestamp.
NFR-17	Cost	The per-incident investigation cost shall be tracked and displayed; the system shall use a low-cost model for routine tasks to keep average cost below $1 per incident during evaluation.

---

6. Use Cases

UC-1: Automated Incident from CI Failure

Actor: GitHub Actions (system)
Precondition: A webhook is configured for the repository.
Main flow: CI fails → GitHub sends signed webhook → n8n receives and calls ingestion API → incident created → orchestrator starts investigation → agents run in parallel → hypothesis ranked → reproduction test written → fix generated → sandbox verifies → developer notified → developer approves → draft PR created → Jira updated → Slack notified.

UC-2: Review and Approve a Fix

Actor: Developer

Precondition: An incident is in READY_FOR_APPROVAL state.
Main flow: Developer opens incident → reads ranked hypotheses and evidence links → reviews causal timeline → checks diff and test results side by side → approves with reason → system creates draft PR and records the decision.
Alternate flow: Developer rejects, adds feedback → system stores feedback in knowledge base → confidence scores updated.

UC-3: Active Evidence Gathering

Actor: Orchestrator (system)
Precondition: Initial evidence collection is insufficient (confidence below threshold).
Main flow: Orchestrator detects insufficient evidence → identifies the most informative source → autonomously fetches additional logs or database rows → re-evaluates hypotheses.
Alternate flow: Additional fetching is not possible → incident transitions to NEEDS_HUMAN_INPUT → developer receives targeted question in dashboard.

UC-4: Hypothesis Testing with Execution Proof

Actor: Orchestrator + Sandbox (system)
Precondition: At least one top-ranked hypothesis exists.
Main flow: Orchestrator selects the top hypothesis → designs an experiment (git bisect, targeted input, boundary test) → sandbox executes the experiment → result confirms or rejects the hypothesis → Verifier updates the hypothesis status to verified or rejected-by-experiment.

UC-5: Investigate a Database-Related Failure

Actor: Database Agent (system)
Precondition: Triage identifies the incident as possibly data-related.
Main flow: Agent reads schema → finds columns changed near incident time → checks for NULLs, constraint violations → masks sensitive values → returns structured findings with timestamps.

Alternate flow: Connection fails → agent reports failure → investigation continues with other agents.

UC-6: Verify a Fix in the Sandbox

Actor: Sandbox (system)
Precondition: A reproduction test and a proposed patch exist.
Main flow: Sandbox builds at incident commit → runs reproduction test (must fail) → applies patch → runs reproduction test (must pass) → runs full test suite (must not regress) → marks fix verified.
Alternate flow: Timeout → fix marked unverified → reason reported to developer.

UC-7: Flaky Test Detected

Actor: Sandbox (system)
Precondition: A CI failure has triggered an incident.
Main flow: Sandbox re-runs failing tests 5 times → tests show mixed pass/fail → classified as flaky → incident marked flaky-only → full pipeline skipped → developer notified → flaky test stored in knowledge base.

UC-8: Run an Evaluation

Actor: Evaluator (team member)
Precondition: Test cases with ground truth are loaded.
Main flow: Evaluator selects test set and baselines → starts evaluation run → system runs full pipeline on each case → computes all metrics → generates comparison tables and charts → evaluator exports for thesis.

UC-9: Jira-Triggered Investigation

Actor: Jira (system)
Precondition: Jira webhook is configured.
Main flow: Developer creates a Jira bug ticket → Jira sends webhook → n8n routes to ingestion API → incident created linked to Jira issue →

investigation runs → on resolution, Jira issue status updated automatically.

---

7. Domain Data Model

These schemas must be frozen by the end of Month 1 before parallel development begins. All modules share these contracts.

Incident

```
id              UUID, primary key
source          ENUM (github_actions, jira, sentry, manual
title           string
description     text
severity        ENUM (critical, high, medium, low)
status          ENUM (received, validated, triaged, invest
                      evidence_collected, hypothesis_ready
                      uncertain, reproduced, fix_generated
                      sandbox_testing, ready_for_approval,
                      rejected, draft_pr_created, resolved
                      failed, needs_human_input, flaky)
repository_url string
commit_sha      string (nullable)
created_at      timestamp
updated_at      timestamp
investigation_cost_usd  decimal (nullable)
jira_issue_id   string (nullable)
github_pr_url   string (nullable)
```

Evidence

```
id              UUID, primary key
incident_id     UUID, foreign key → Incident
```

```
type            ENUM (code, log, database, commit, test_re
                      historical_incident, schema_change)
source          string (file path, URL, table name, etc.)
locator         string (line number, commit hash, row ID)
content         text (masked if sensitive)
content_hash    string (for deduplication)
timestamp       timestamp (when the evidence event occurre
relevance_score decimal (assigned by RAG reranking)
created_at      timestamp
```

Claim

```
id              UUID, primary key
incident_id     UUID, foreign key → Incident
hypothesis_id   UUID, foreign key → Hypothesis (nullable)
statement       text
evidence_ids    UUID[] (must not be empty for a supported
confidence      decimal (0-1)
status          ENUM (supported, weak, contradicted, rejec
                      verified_by_experiment)
verifier_reason text (reason for rejection, if applicable
```

Hypothesis

```
id                     UUID, primary key
incident_id            UUID, foreign key → Incident
statement              text
supporting_evidence    UUID[]
contradicting_evidence UUID[]
score                  decimal (0-1)
status                 ENUM (candidate, verified, rejected
                           rejected_by_experiment, uncert
experiment_type        string (nullable; e.g., git_bisect,
experiment_result      text (nullable)
```

Fix

```
id              UUID, primary key
incident_id     UUID, foreign key → Incident
branch          string
diff            text
files_changed   string[]
rationale       text
status          ENUM (generated, sandbox_verified, approve
                      rejected, pr_created)
approved_by     UUID (nullable, foreign key → User)
approved_at     timestamp (nullable)
rejection_reason_category  ENUM (wrong_cause, incomplete_
                                 style, other) (nullable
```

SandboxRun

```
id              UUID, primary key
incident_id     UUID, foreign key → Incident
fix_id          UUID, foreign key → Fix (nullable)
docker_image    string
command         string
cpu_limit       string
memory_limit    string
timeout_sec     integer
exit_code       integer
stdout_ref      string (storage path)
stderr_ref      string (storage path)
status          ENUM (running, passed, failed, timeout,
                      build_error, flaky)
started_at      timestamp
finished_at     timestamp
```

TestResult

```
id              UUID, primary key
sandbox_run_id UUID, foreign key → SandboxRun
test_name       string
phase           ENUM (before_patch, after_patch, regressio
run_number      integer (for flaky rerun tracking)
status          ENUM (passed, failed, error, skipped)
duration_ms     integer
failure_message text (nullable)
```

---

8. Security Requirements

ID	Requirement
SEC-1	All API endpoints shall require authentication (JWT or session token). Users shall only access projects they are members of.
SEC-2	All webhook endpoints shall validate HMAC signatures before processing the payload. Invalid or missing signatures shall return HTTP 401.
SEC-3	Duplicate webhook events shall be detected via an idempotency key and silently acknowledged without reprocessing.
SEC-4	The Database Agent shall connect using a read-only database account. A query inspection layer shall reject any non-SELECT statement before it reaches the database.
SEC-5	Sandbox containers shall run as a non-privileged user. Network egress shall be blocked. Host filesystem shall not be mounted. No production credentials shall be passed into the container.
SEC-6	The Fix Agent shall write only to isolated branches or worktrees. It shall have no access to the main branch and no push permissions.

SEC-7	Repository files, issue text, log content, and Sentry payloads shall be treated as untrusted input. Prompt construction shall not allow injected content to override system instructions.
SEC-8	All credentials, tokens, and API keys shall be stored encrypted at rest. They shall never be returned by any API after initial save. They shall never be committed to version control.
SEC-9	All agent actions, tool calls, evidence items created, sandbox runs, and human approval/rejection decisions shall be written to an append-only audit log with user identity and timestamp.
SEC-10	When privacy mode is enabled, no source code, log content, or database data shall be transmitted to external LLM providers. All inference shall use the locally hosted model.
SEC-11	PII patterns (email addresses, phone numbers, passwords, tokens) in logs and database results shall be masked with a configurable redaction pattern before storage or LLM transmission.

9. Constraints and Assumptions

Constraints

- Python projects are fully supported. JavaScript/TypeScript projects are fully supported. No other languages in scope.
- Database access is read-only. The system shall never write to a target database.
- Automatic deployment to production is prohibited. Human approval is always required.
- Development uses free-tier LLM models (Gemini Flash, Groq) where possible. External paid APIs are used only for final

evaluation and demo.
- The project timeline is 9 months with a 3-person team. Optional items are cut before Must items if time runs short.

Assumptions

- The target application uses PostgreSQL.
- The user provides a repository URL accessible via GitHub API and a read-only database connection string.
- Docker is available on the machine running the platform.
- A Jira project (free plan minimum) is available.
- The sample application for testing uses injected, deterministic bugs with known ground truth.

---

10. Out of Scope

The following are explicitly outside this project's scope. They may be addressed in future work.

Item	Reason
Automatic deployment to production	Human approval is always required by design
Training or fine-tuning a foundation model	Requires resources and data beyond FYP scope
Java, Go, C++, or other languages	Modular adapter architecture allows future addition
MySQL, MongoDB, or other databases	PostgreSQL only; adapter pattern allows future addition
Slack bot as a primary client	Slack is a notification target only
Full CLI interface	Web dashboard is the primary interface

Multi-tenant billing and subscription management	Post-FYP commercialization concern
Real-time performance profiling (APM)	Separate product category
Chaos engineering / fault injection	Post-FYP extension

---

11. Glossary

Term	Definition
APM	Application Performance Monitoring
CI/CD	Continuous Integration / Continuous Deployment
FYP	Final Year Project
HMAC	Hash-based Message Authentication Code (used for webhook signature verification)
MTTD	Mean Time to Detect — average time from incident occurrence to detection
MTTR	Mean Time to Resolve — average time from detection to resolution
Ollama	A tool for running large language models locally
PII	Personally Identifiable Information
RAG	Retrieval-Augmented Generation
SRS	Software Requirements Specification
SRE	Site Reliability Engineer
tree-sitter	A parser generator tool used for code analysis
vLLM	A high-throughput inference engine for locally hosted LLMs

WebSocket	A protocol for full-duplex communication used for live agent status streaming

---

End of SRS — Version 2.0
Next revision trigger: supervisor review, Month 1 schema freeze, or scope change decision.